

30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 3)

“Scheduling and
Core Affinity”



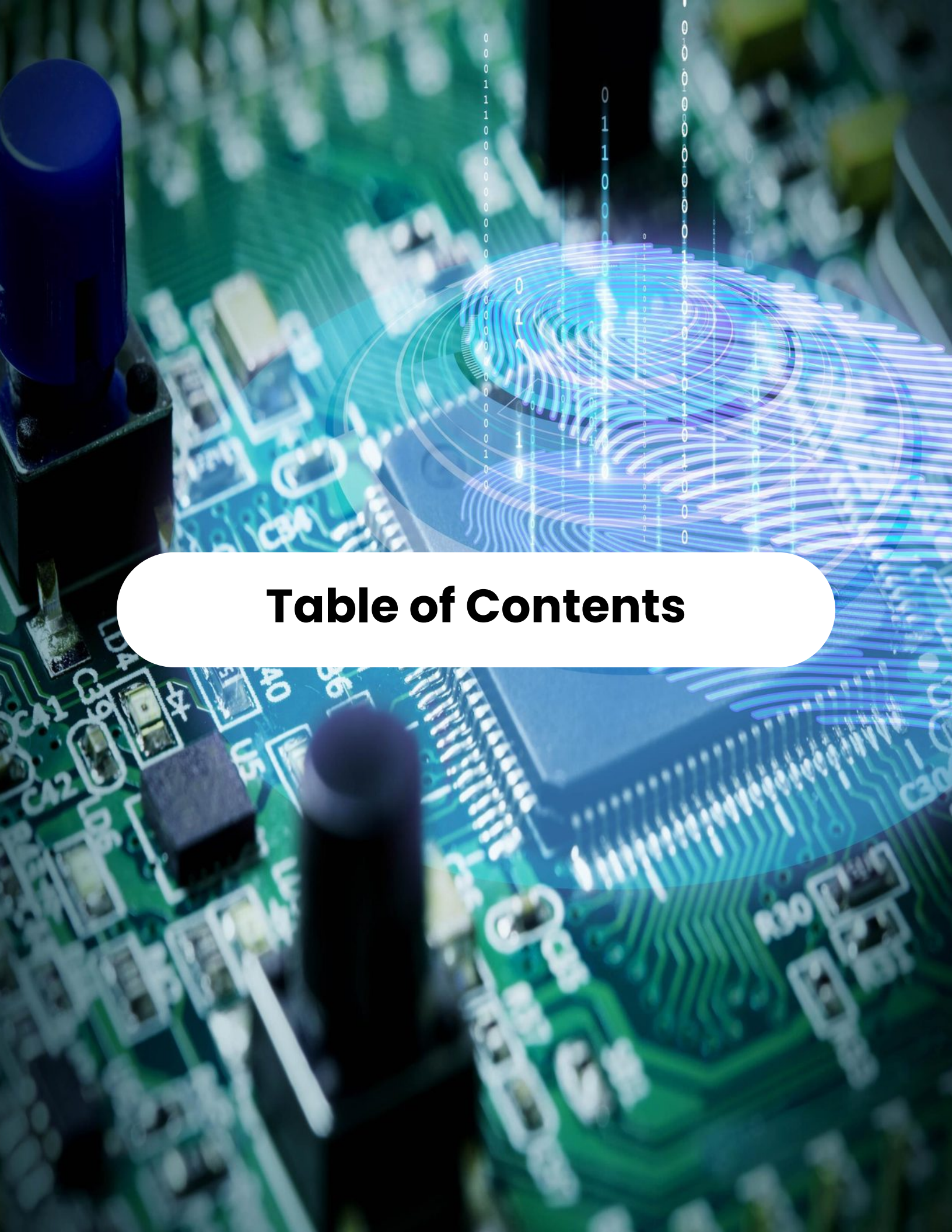
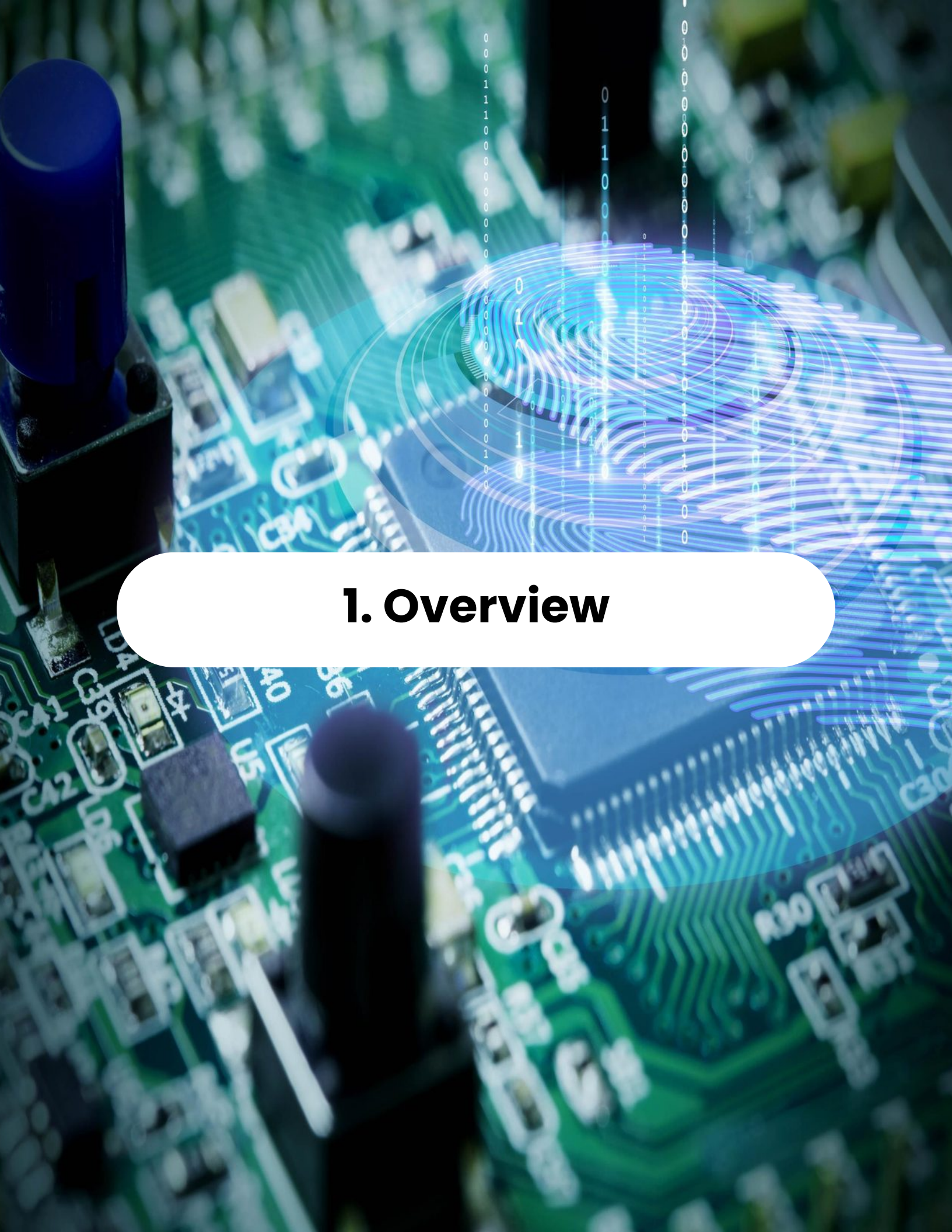


Table of Contents

Table of Contents

1. Overview
2. What is Task Scheduling?
3. Task States
4. Priorities and Preemption
5. ESP32 Dual-Core Scheduling (SMP)
6. Core Affinity APIs
7. Code Walkthrough
8. When to Pin (and When Not To)
9. Summary
10. Challenge for Today



1. Overview

1. Overview

On Day 3, you'll learn how to:

- Explain **how the FreeRTOS scheduler** decides which task runs next
- Describe the four **task states** and what moves a task between them
- Use **priorities, preemption, and time slicing** to control behavior
- Understand how the ESP32's **two cores** share scheduling work (SMP)
- Create **unpinned** tasks with `xTaskCreate()` and **pinned** tasks with `xTaskCreatePinnedToCore()`
- Verify which core a task runs on using `xPortGetCoreID()`

By the end of this lesson, you'll know not only how to start a task, but also **where and when it runs** — the foundation for every multitasking design you'll build on the ESP32.

Prerequisites: Day 1 (RTOS and FreeRTOS basics) and Day 2 (a working ESP-IDF + VS Code setup that can build, flash, and monitor a project).



2. What is Task Scheduling?

2. What is Task Scheduling?

The **scheduler** is the part of the FreeRTOS kernel that decides, at every moment, **which task gets the CPU**.

A CPU core can execute only one instruction stream at a time. The scheduler creates the illusion of many things happening at once by switching between tasks very quickly.

A context switch:

- Saves the CPU registers and stack pointer of the task being paused
- Restores the registers and stack pointer of the task being resumed
- Happens in microseconds, so tasks appear to run in parallel

Because each task has its **own stack**, it continues exactly where it left off — it never knows it was paused.

2. What is Task Scheduling?

When does the scheduler run?

The scheduler is invoked whenever the set of runnable tasks might have changed:

- On every **tick interrupt** (a periodic timer)
- When a task **blocks** (e.g., `vTaskDelay()`)
- When a task becomes ready because a **delay expires** or an event arrives
- When a task explicitly **yields** with `taskYIELD()`
- When a task is **created, deleted, suspended, or resumed**

Remember: You do not call the scheduler yourself. You influence it through task priorities, delays, and blocking calls.

2. The System Tick

FreeRTOS keeps time with a periodic **tick interrupt**. The tick frequency is set by `configTICK_RATE_HZ` .

In ESP-IDF the default is **100 Hz**, so **1 tick = 10 ms** (you saw this value in the menuconfig screenshot on Day 2).

Milliseconds	Macro call	Ticks at 100 Hz
1000 ms	<code>pdMS_TO_TICKS(1000)</code>	100
500 ms	<code>pdMS_TO_TICKS(500)</code>	50
20 ms	<code>pdMS_TO_TICKS(20)</code>	2
10 ms	<code>pdMS_TO_TICKS(10)</code>	1
5 ms	<code>pdMS_TO_TICKS(5)</code>	0 (rounds down!)

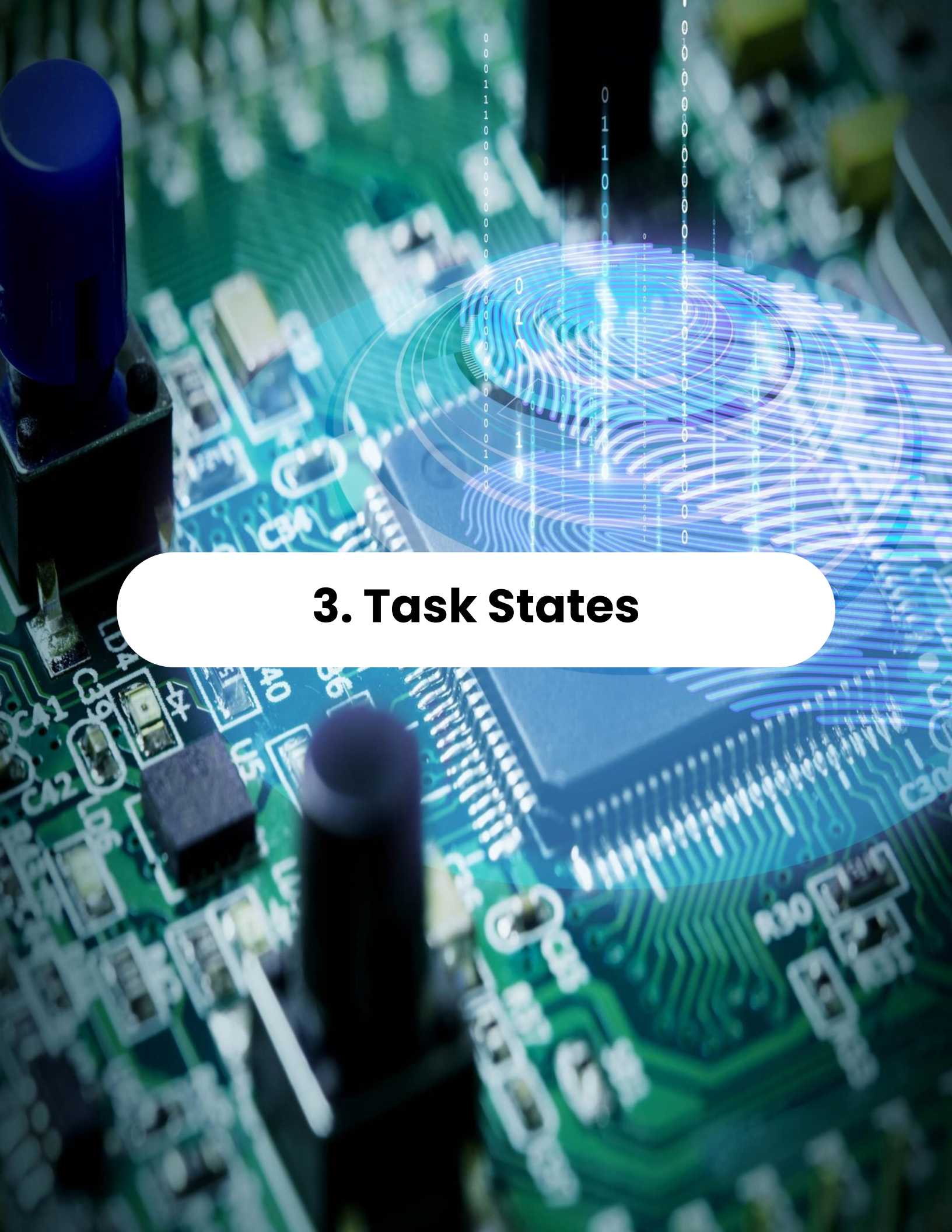
Careful: Delays are measured in whole ticks. At 100 Hz, any delay shorter than 10 ms becomes 0 ticks and the task just yields. Need finer timing? Raise the tick rate in menuconfig (Component config → FreeRTOS → Kernel).

2. Scheduling Policy in FreeRTOS

ESP-IDF runs FreeRTOS with **fixed-priority preemptive scheduling with time slicing**. In plain words, three rules:

- **Rule 1 — Highest priority wins.** The Ready task with the highest priority is always the one that runs.
- **Rule 2 — Preemption.** If a higher-priority task becomes Ready, it takes the CPU **immediately**, interrupting the lower one.
- **Rule 3 — Time slicing.** Ready tasks that share the **same priority** take turns, switching on each tick (round-robin).

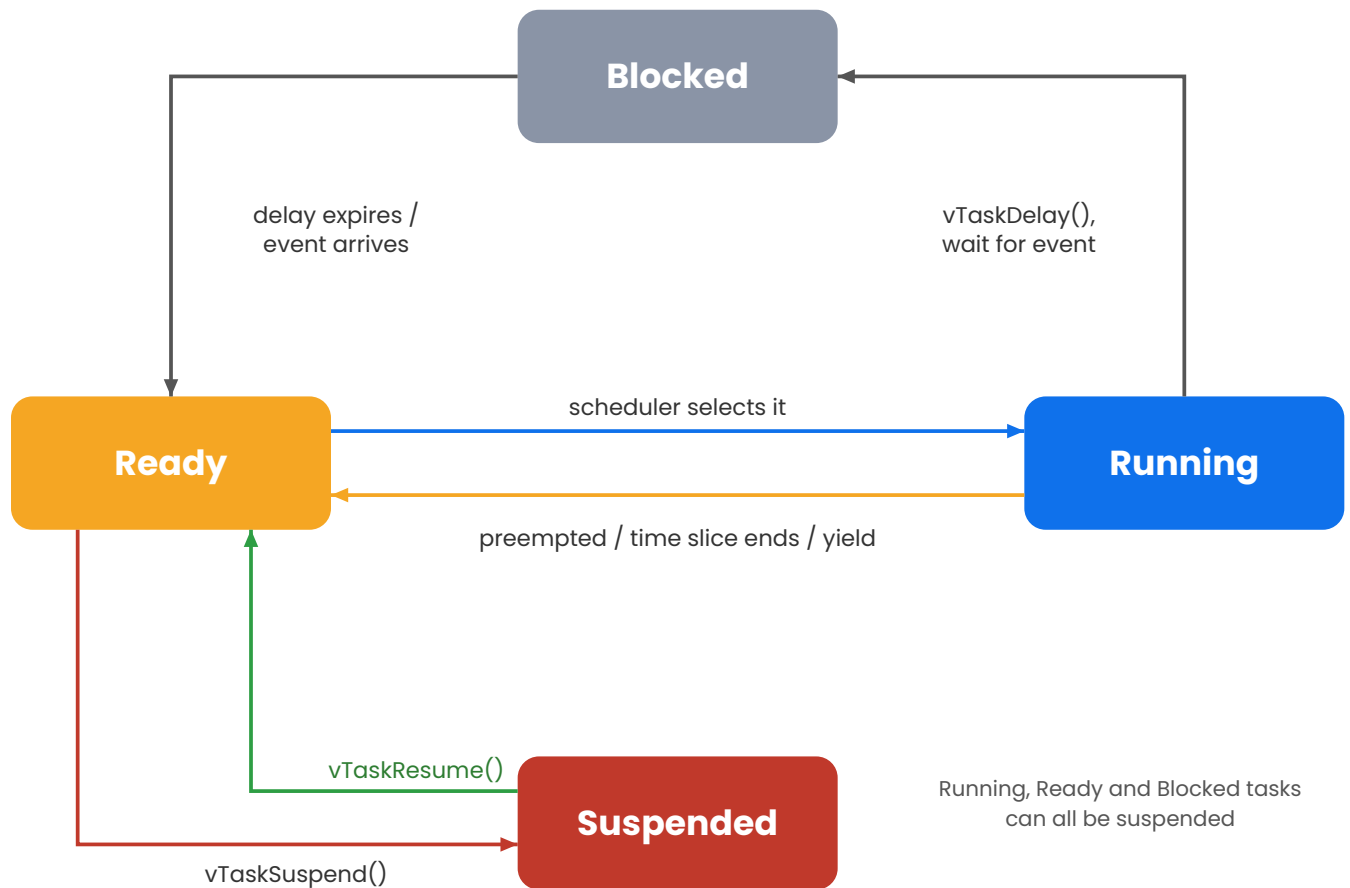
Day 1 mentioned that an RTOS can be preemptive or cooperative. ESP-IDF uses **preemptive** scheduling, so tasks never need to “give up” the CPU voluntarily for higher-priority work to run.



3. Task States

3. Task States

At any moment, every FreeRTOS task is in exactly one of four states:



Only **one task per core** can be in the Running state at a time.

3. The Four States Explained

State	Meaning	Typical cause
Running	Currently executing on a CPU core	Scheduler picked it
Ready	Able to run, but a higher-priority (or same-priority) task has the core	Preempted, or just unblocked
Blocked	Waiting for time or an event; uses no CPU	<code>vTaskDelay()</code> , queue/semaphore wait with timeout
Suspended	Excluded from scheduling until resumed	<code>vTaskSuspend()</code>

Blocked is the state you want most tasks to spend most of their time in. A task that blocks hands the CPU to others — and lets the core go idle to save power.

3. Moving Between States

What happens	Transition	API / cause
Task calls a delay	Running → Blocked	<code>vTaskDelay()</code> , <code>vTaskDelayUntil()</code>
Delay time elapses	Blocked → Ready	Tick interrupt
Higher-priority task wakes up	Running → Ready	Preemption
Scheduler assigns the core	Ready → Running	Scheduler
Task is paused by code	Any → Suspended	<code>vTaskSuspend()</code>
Task is un-paused	Suspended → Ready	<code>vTaskResume()</code>
Task ends itself or is removed	Any → Deleted	<code>vTaskDelete()</code>

In our example: Each task prints, calls `vTaskDelay(pdMS_TO_TICKS(1000))` , and becomes Blocked for ~1 second. It spends almost its entire life Blocked.

3. The Idle Task (One per Core)

When every other task is Blocked or Suspended, something still has to run. FreeRTOS creates an **Idle task** at priority **0** on each core. On the ESP32 they are named `IDLE0` and `IDLE1` .

What the Idle task does:

- Frees the memory of tasks that were deleted
- Lets the core enter a low-power state when there is no work
- Feeds the **Task Watchdog Timer** (enabled by default for the idle tasks)

Watchdog warning: If a task at a higher priority loops forever without ever blocking, the Idle task never runs and you will see “Task watchdog got triggered” in the monitor (default timeout: 5 s). Always include a `vTaskDelay()` or another blocking call in task loops.



4. Priorities and Preemption

4. Task Priorities

Every task gets a priority when it is created — the 5th argument of `xTaskCreate()` .

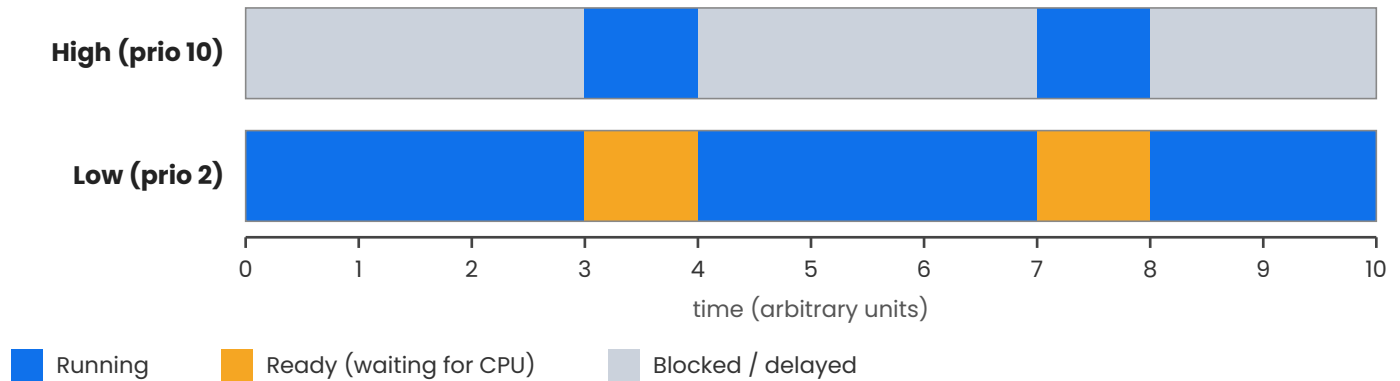
- **Higher number = higher priority**
- ESP-IDF range: `0` to `configMAX_PRIORITIES - 1` (that is **0 to 24**)
- Priority `0` is reserved for the Idle task
- The `app_main()` task (called `main`) runs at priority **1**
- Many ESP-IDF system tasks (Wi-Fi, `esp_timer` , IPC) run at high priorities

Priority	Typical use
0	Idle tasks (do not use)
1 – 5	Normal application work, logging, UI, slow sensors
6 – 15	Time-sensitive processing, communication handling
16 – 24	Reserved in practice for system-critical tasks — avoid

Tip: Start your application tasks around priority 3–5 and only raise a task when you can explain **why** it must preempt others.

4. Preemption in Action

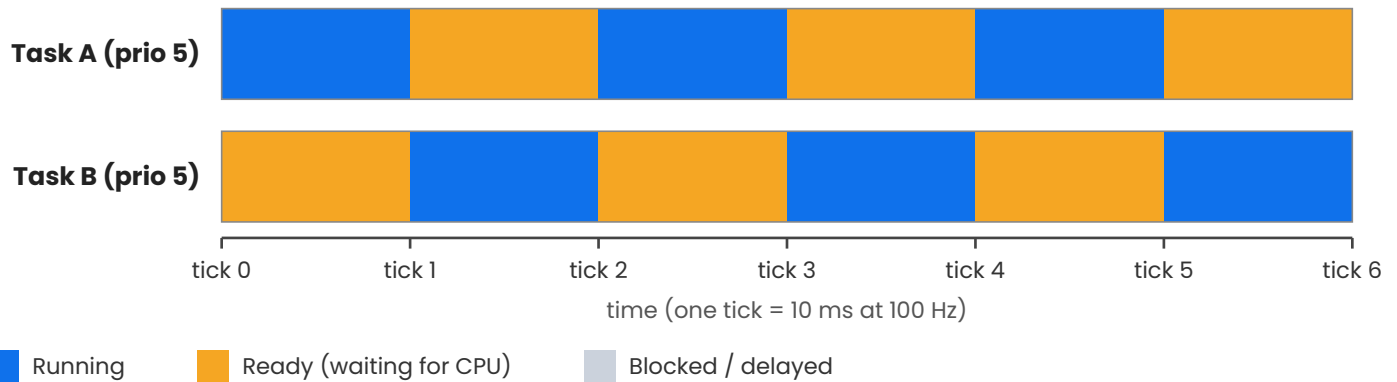
Imagine one core with two tasks: a **High**-priority task that wakes up periodically, and a **Low**-priority task that is always busy.



- At **t = 3** the High task's delay expires. It becomes Ready and **instantly preempts** Low.
- When High blocks again (`vTaskDelay()`), Low goes back to Running.
- Low never decides to stop — the kernel does it for it.

4. Equal Priorities and Time Slicing

What if two tasks have the **same priority** and both want the CPU on a single core?

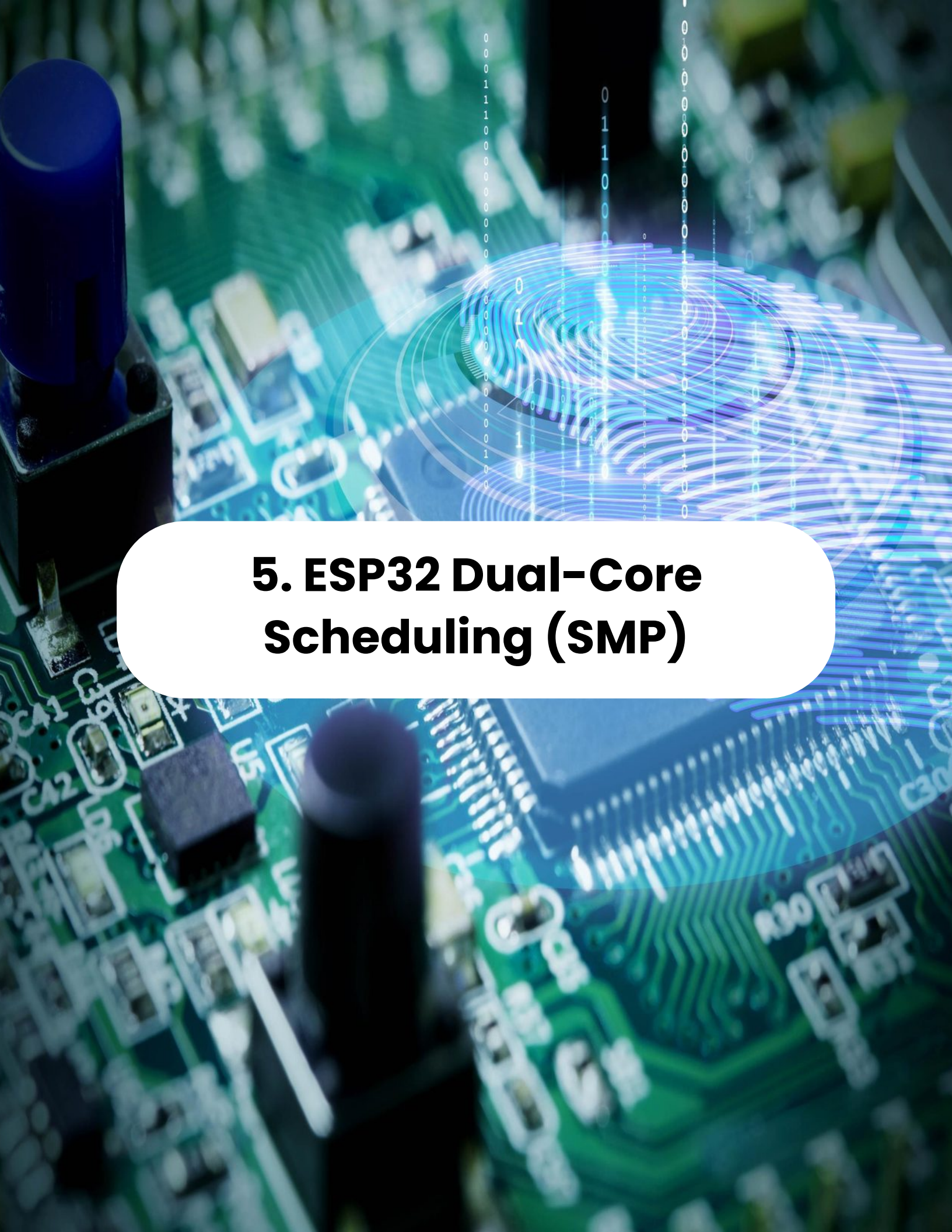


- The scheduler alternates between them **every tick** (round-robin).
- In today's example both tasks use priority **5**, but they are almost always Blocked, so they rarely compete.
- On a **dual-core** ESP32, two Ready tasks can truly run **at the same time** — one per core.

4. Priority Design Guidelines

- **Tighter deadline → higher priority.** A task reacting to a sensor interrupt outranks a task updating a display.
- **Always block.** High-priority tasks should do their work quickly and then wait (delay, queue, semaphore).
- **Avoid starvation.** A busy high-priority task can prevent lower ones from ever running.
- **Keep it simple.** A handful of distinct priorities is easier to reason about than 20 different ones.
- **Priority is not speed.** It only decides who is chosen first; it does not make code run faster.

Coming later: Priority inversion and how mutexes handle it will be covered when we reach synchronization.



5. ESP32 Dual-Core Scheduling (SMP)

5. Two Cores, One FreeRTOS

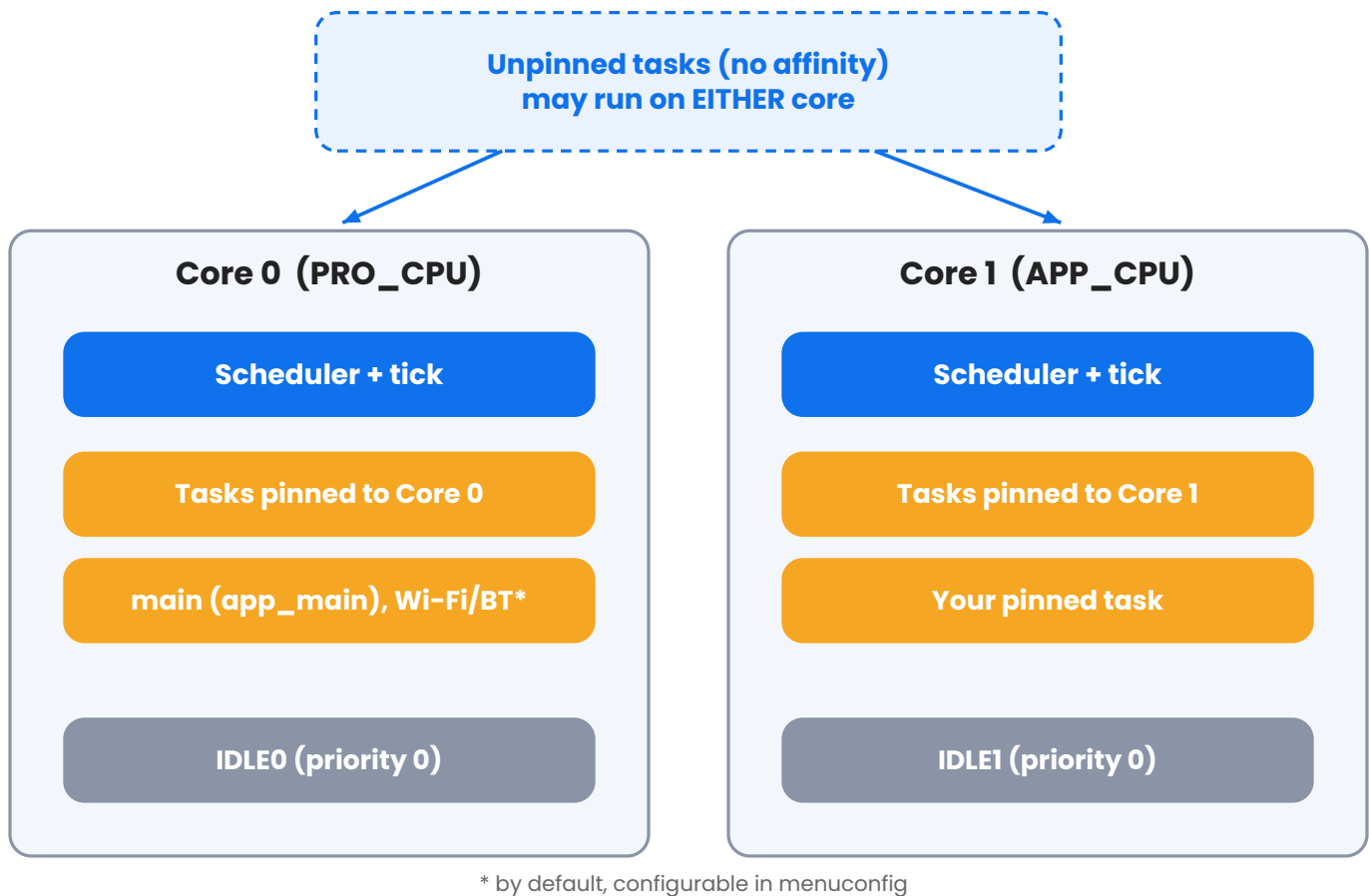
The classic ESP32 has two Xtensa cores, named by Espressif:

- **Core 0 — PRO_CPU** (protocol CPU)
- **Core 1 — APP_CPU** (application CPU)

Both cores share the same memory and peripherals and run **one** FreeRTOS instance in **Symmetric Multiprocessing (SMP)** mode:

- Each core has its **own tick interrupt** and its **own Idle task** (`IDLE0` , `IDLE1`)
- At each scheduling point, a core picks the **highest-priority Ready task that is allowed to run on it**
- Two tasks can execute at the **same instant**, one on each core
- A task is either **pinned** to one core or **unpinned** (free to use either)

5. How Tasks Land on Cores



Pinned tasks are only considered by their own core. Unpinned tasks are offered to **both** cores — whichever core is ready to schedule first (and has no higher-priority work) takes them.

5. Who Runs Where by Default?

Task	Core (default)	Notes
<code>main</code> (runs <code>app_main()</code>)	Core 0	Priority 1; ends when <code>app_main()</code> returns
<code>IDLE0</code> / <code>IDLE1</code>	One per core	Priority 0, created by the kernel
<code>ipc0</code> / <code>ipc1</code>	One per core	Inter-processor calls between cores
<code>esp_timer</code>	Core 0	High-resolution timer callbacks
Wi-Fi / Bluetooth tasks	Core 0	Can be moved via menuconfig
Your unpinned tasks	Either	Scheduler decides at runtime

Why it matters: Because system and wireless work lives on Core 0 by default, Core 1 is often the quieter place for time-sensitive application work. That is one reason today's example pins a task to Core 1.

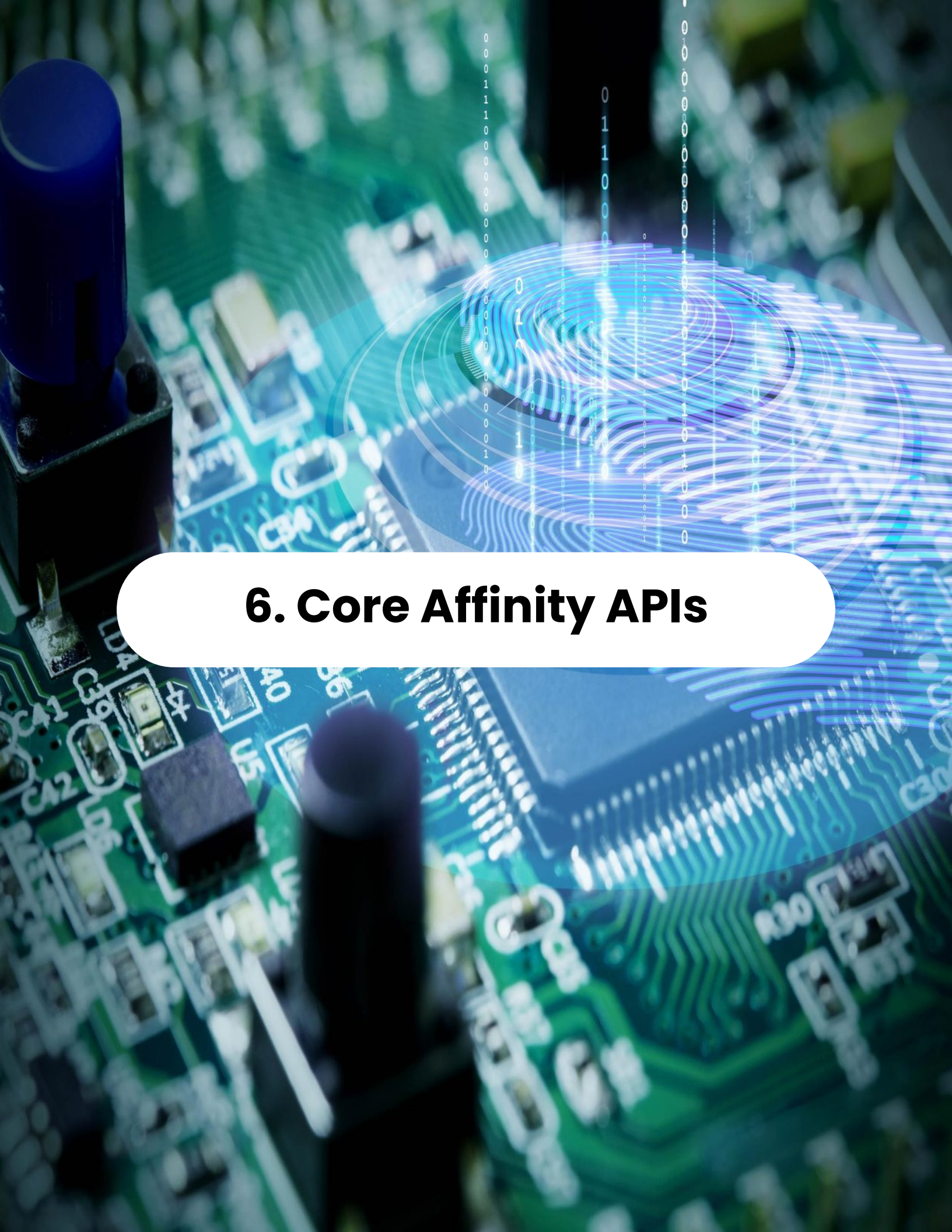
5. Is My Chip Dual-Core?

Not every ESP32 variant has two cores. **Core 1 only exists on dual-core chips.**

Dual-core	Single-core
ESP32	ESP32-S2
ESP32-S3	ESP32-C2, C3, C5, C6
ESP32-P4	ESP32-H2

Heads-up: The module on this course's cover art is an **ESP32-C6, which is single-core**. On such chips there is only Core 0, and pinning a task to Core 1 will fail. We show a portable pattern for this in Section 8.

Check your own board's datasheet or the chip line printed at boot in the monitor output before running the pinned-task example.



6. Core Affinity APIs

6. What is Core Affinity?

Core affinity is the set of cores a task is allowed to run on.

Affinity	Meaning
Pinned to Core 0	Runs only on Core 0 (every time it becomes Ready)
Pinned to Core 1	Runs only on Core 1
Unpinned (<code>tskNO_AFFINITY</code>)	Can run on any core; the scheduler decides

Two FreeRTOS functions in ESP-IDF create tasks, and they differ **only** in how they treat affinity:

- `xTaskCreate()` → creates an **unpinned** task
- `xTaskCreatePinnedToCore()` → lets **you choose** the core (or none)

Good to know: In ESP-IDF, `xTaskCreate()` is simply `xTaskCreatePinnedToCore()` called with `tskNO_AFFINITY`.

6. xTaskCreate()

```
1 BaseType_t xTaskCreate(  
2     TaskFunction_t pvTaskCode,      // Function the task runs  
3     const char * const pcName,      // Human-readable name  
4     const uint32_t usStackDepth,    // Stack size in BYTES (ESP-IDF)  
5     void * const pvParameters,     // Argument passed to the task  
6     UBaseType_t uxPriority,         // 0 (lowest) .. 24 (highest)  
7     TaskHandle_t * const pxCreatedTask // Optional handle (or NULL)  
8 );
```

- Returns `pdPASS` on success, or an error code (e.g., not enough memory)
- The task is **unpinned** — it may run on either core
- **ESP-IDF difference:** vanilla FreeRTOS counts stack depth in **words**; ESP-IDF counts in **bytes**. That is why `2048` in our examples means 2 KB.
- The task function must **never return** — use an infinite loop, or call `vTaskDelete(NULL)` when done

6. xTaskCreatePinnedToCore()

```
1 BaseType_t xTaskCreatePinnedToCore(  
2     TaskFunction_t pvTaskCode,  
3     const char * const pcName,  
4     const uint32_t usStackDepth,  
5     void * const pvParameters,  
6     UBaseType_t uxPriority,  
7     TaskHandle_t * const pxCreatedTask,  
8     const BaseType_t xCoreID           // <-- the extra argument  
9 );
```

Identical to `xTaskCreate()` plus one final argument, `xCoreID` :

xCoreID	Result
0	Pinned to Core 0 (PRO_CPU)
1	Pinned to Core 1 (APP_CPU) — dual-core chips only
tskNO_AFFINITY	Not pinned — behaves like <code>xTaskCreate()</code>

Today's call: `xTaskCreatePinnedToCore(task_pinned_core1, "Task Core1", 2048, NULL, 5, NULL, 1);`

6. Asking: “Which Core Am I On?”

Inside a running task, call `xPortGetCoreID()` to learn the core currently executing it:

```
1 void my_task(void *pvParameters) {  
2     while (1) {  
3         int core = xPortGetCoreID();    // returns 0 or 1  
4         printf("Running on Core %d\n", core);  
5         vTaskDelay(pdMS_TO_TICKS(1000));  
6     }  
7 }
```

- Returns `0` for Core 0 and `1` for Core 1
- ESP-IDF also provides `esp_cpu_get_core_id()` for the same purpose
- The answer reflects **where the code is running right now** — for an unpinned task it may differ over time

6. Quick API Reference

Function	Purpose
<code>xTaskCreate()</code>	Create an unpinned task (dynamic memory)
<code>xTaskCreatePinnedToCore()</code>	Create a task pinned to Core 0 / Core 1 / any
<code>xTaskCreateStaticPinnedToCore()</code>	Same, but you supply the stack and TCB memory
<code>xPortGetCoreID()</code>	Core currently executing the caller
<code>vTaskDelay()</code>	Block the calling task for N ticks
<code>pdMS_TO_TICKS()</code>	Convert milliseconds to ticks
<code>vTaskDelete()</code>	Delete a task (<code>NULL</code> = the calling task)
<code>tskNO_AFFINITY</code>	Constant meaning "any core"

Tip: Wondering which function names exist in your ESP-IDF version? In VS Code, hover over a function name or press F12 to jump into the FreeRTOS headers shipped with ESP-IDF.



7. Code Walkthrough

7. Today's Example Project

The program creates **two tasks that report which core they run on**:

- **Task 1 — Unpinned:** created with `xTaskCreate()` . Any core may run it.
- **Task 2 — Pinned:** created with `xTaskCreatePinnedToCore()` , always on **Core 1**.
- Both print a line every **1 second** using `vTaskDelay()` .

Set it up like Day 2:

- Press `Ctrl+Shift+P` → **ESP-IDF: Create project using template**
- Name it, e.g., `task_core_affinity` , and choose **template-app**
- Replace the contents of `main/main.c` with the code on the next page
- Make sure the target matches a **dual-core** chip (e.g., ESP32 or ESP32-S3)

7. The Complete Code

```
1 #include <stdio.h>
2 #include "freertos/FreeRTOS.h"
3 #include "freertos/task.h"
4
5 void task_unpinned(void *pvParameters) {
6     while (1) {
7         printf("Unpinned Task running on Core %d\n", xPortGetCoreID());
8         vTaskDelay(pdMS_TO_TICKS(1000));
9     }
10 }
11
12 void task_pinned_core1(void *pvParameters) {
13     while (1) {
14         printf("Pinned Task running on Core %d\n", xPortGetCoreID());
15         vTaskDelay(pdMS_TO_TICKS(1000));
16     }
17 }
18
19 void app_main(void) {
20     // Task without core affinity (runs on any available core)
21     xTaskCreate(task_unpinned, "Task Unpinned", 2048, NULL, 5, NULL);
22
23     // Task pinned to Core 1
24     xTaskCreatePinnedToCore(task_pinned_core1, "Task Core1", 2048, NULL, 5, NULL, 1);
25 }
```

Note: The original homework file named the second task “Task Core0” and described it as pinned to Core 0 in a comment. It is actually pinned to **Core 1**, so the name above was corrected to avoid confusion.

7. The Task Functions

```
1 void task_unpinned(void *pvParameters) {
2     while (1) {
3         printf("Unpinned Task running on Core %d\n", xPortGetCoreID());
4         vTaskDelay(pdMS_TO_TICKS(1000));
5     }
6 }
```

- `void *pvParameters` — every task function takes one `void *` argument. We pass `NULL`, so it is unused.
- `while (1)` — tasks run forever; a task function must not simply return.
- `xPortGetCoreID()` — captures the core executing this line **at this moment**.
- `vTaskDelay(pdMS_TO_TICKS(1000))` — blocks the task for 100 ticks. The core is free to run other tasks or idle.
- `task_pinned_core1` has the **same body** — the only difference between the two tasks is how they are created.

7. Creating the Tasks in app_main()

```
1 void app_main(void) {
2     xTaskCreate(task_unpinned, "Task Unpinned", 2048, NULL, 5, NULL);
3
4     xTaskCreatePinnedToCore(task_pinned_core1, "Task Core1",
5                             2048, NULL, 5, NULL, 1);
6 }
```

Argument	Value	Meaning
Function	task_unpinned	Code the task executes
Name	"Task Unpinned"	Shown in debug tools and logs
Stack	2048	2048 bytes of private stack
Parameter	NULL	Nothing passed to the task
Priority	5	Both tasks equal priority
Handle	NULL	We do not need to control it later
Core (2nd call)	1	Pinned to Core 1

When `app_main()` returns, its `main` task is deleted — the two new tasks keep running on their own.

7. Build, Flash, and Monitor

Use the same workflow as Day 2:

- **Build:** click **Build Project** (or `Ctrl+Shift+P` → *ESP-IDF: Build your project*)
- **Flash:** connect the board by USB, then click **Flash Device**
- **Monitor:** click **Monitor Device** to see the output
- Or click **ESP-IDF: Build, Flash and Monitor** to do all three in one step

You should see lines like these, repeating every second:

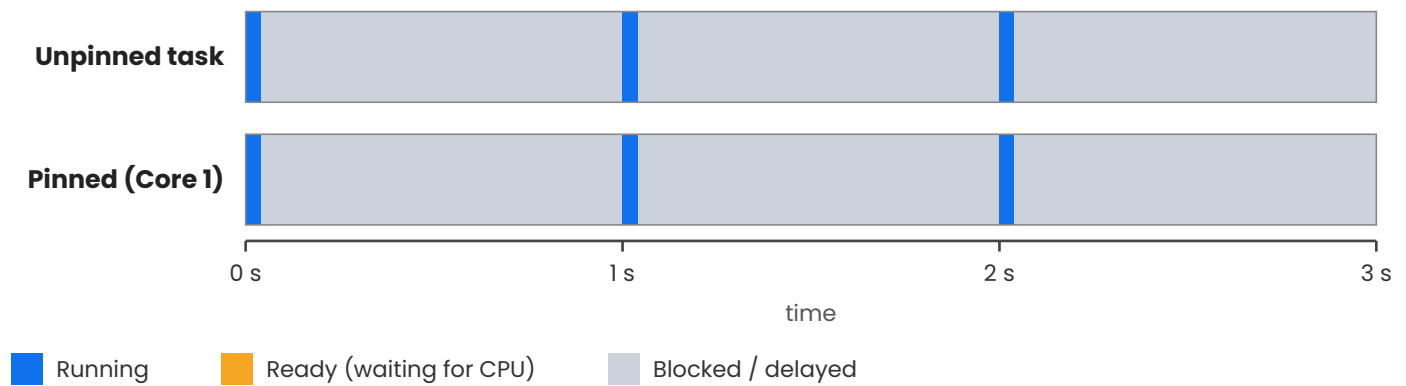
A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The text inside the terminal is white and shows a repeating pattern of task scheduling information.

```
Unpinned Task running on Core 0  
Pinned Task running on Core 1  
Unpinned Task running on Core 0  
Pinned Task running on Core 1  
...
```

Note: The core reported by the **unpinned** task can be 0 or 1 — the scheduler chooses, and the result can differ between runs and ESP-IDF versions. The **pinned** task must always report Core 1.

7. Reading the Output

Both tasks print once per second. On the timeline, each task runs for a tiny instant and then stays Blocked:



- Each task spends **well under 1%** of its time actually running.
- The two tasks are not waiting for each other; with two cores they may print **at nearly the same instant**.
- Because both prints happen almost together, the **order** of the two lines can change from one second to the next.

7. Gotchas in the Example

- **Single-core targets:** `xTaskCreatePinnedToCore(..., 1)` has no Core 1 to use on an ESP32-C3/C6/S2. Use a dual-core chip or the portable pattern in Section 8.
- **Names matter:** match task names and comments to the real core. A task called "Core0" pinned to Core 1 makes debugging painful.
- **Stack size:** `printf` uses a fair amount of stack. 2048 bytes works here, but if you add formatting or larger buffers, increase it.
- **Delays are minimums:** `vTaskDelay()` blocks for **at least** the requested time. For exact periods, `vTaskDelayUntil()` is the right tool (later lessons).
- **Interleaved output:** two cores printing at once can interleave on the serial console. Keep log lines short and complete.

8. When to Pin (and When Not To)

8. Default: Let the Scheduler Decide

For most tasks, **do not pin**. An unpinned task lets FreeRTOS balance the load across both cores automatically.

- If one core is busy, an unpinned task can use the other
- Your code stays portable between single-core and dual-core chips
- No risk of overloading one core while the other idles

Pin a task only when you have a **specific reason**.

Rule of thumb: Start unpinned. Measure. Pin only if a measurable problem (jitter, missed deadlines, driver requirement) justifies it.

8. Good Reasons to Pin a Task

- **Driver or interrupt ownership.** Interrupts allocated with `esp_intr_alloc()` belong to the core that registered them. A task that works closely with that interrupt often belongs on the same core.
- **Isolating time-critical work.** Wi-Fi, Bluetooth, and system tasks default to Core 0. Placing a latency-sensitive task on Core 1 keeps it away from that activity.
- **Isolating heavy computation.** Pinning a CPU-hungry task to one core keeps the other core responsive for communication.
- **Predictability.** A fixed core makes timing behavior easier to measure and reason about.
- **Library requirements.** Some libraries document that a task must run on a particular core.

8. Costs and Risks of Pinning

- **Load imbalance.** A pinned task cannot move to an idle core, even if its own core is busy.
- **Less flexibility.** The scheduler has fewer options, which can reduce overall throughput.
- **Portability.** Core 1 does not exist on single-core chips.
- **It is not a speed-up.** Pinning does not make code faster; it only restricts where it can run.
- **Cross-core data sharing needs care.** Two tasks on different cores can run at the exact same time, so shared variables need protection (queues, mutexes, critical sections) — topics for upcoming days.

8. Portable Code for Any Chip

Use the Kconfig symbol `CONFIG_FREERTOS_UNICORE` to pick the right call at compile time:

```
1 void app_main(void) {
2     xTaskCreate(task_unpinned, "Task Unpinned", 2048, NULL, 5, NULL);
3
4     #if CONFIG_FREERTOS_UNICORE
5         // Single-core chip (or "run on first core only" enabled)
6         xTaskCreate(task_pinned_core1, "Task Core1", 2048, NULL, 5, NULL);
7     #else
8         // Dual-core chip: pin to Core 1
9         xTaskCreatePinnedToCore(task_pinned_core1, "Task Core1",
10                                2048, NULL, 5, NULL, 1);
11 #endif
12 }
```

- `CONFIG_FREERTOS_UNICORE` is set on single-core targets and when “Run FreeRTOS only on first core” is enabled
- The same source file now builds correctly on every ESP32 variant

8. Common Misconceptions

Myth	Reality
"A pinned task owns its core."	Other tasks can still run on that core. Priority decides who wins.
"Pinning makes code faster."	It only restricts placement; it never adds performance.
"An unpinned task stays on one core."	No guarantee. It can run on a different core each time it wakes.
"Higher priority runs faster."	It only means the task is chosen first when several are Ready.
" <code>vTaskDelay(1000 ms)</code> is exact."	It is a minimum, rounded to ticks, plus scheduling latency.
"More tasks = more speed."	Each task costs memory and context switches. Create tasks for structure, not speed.

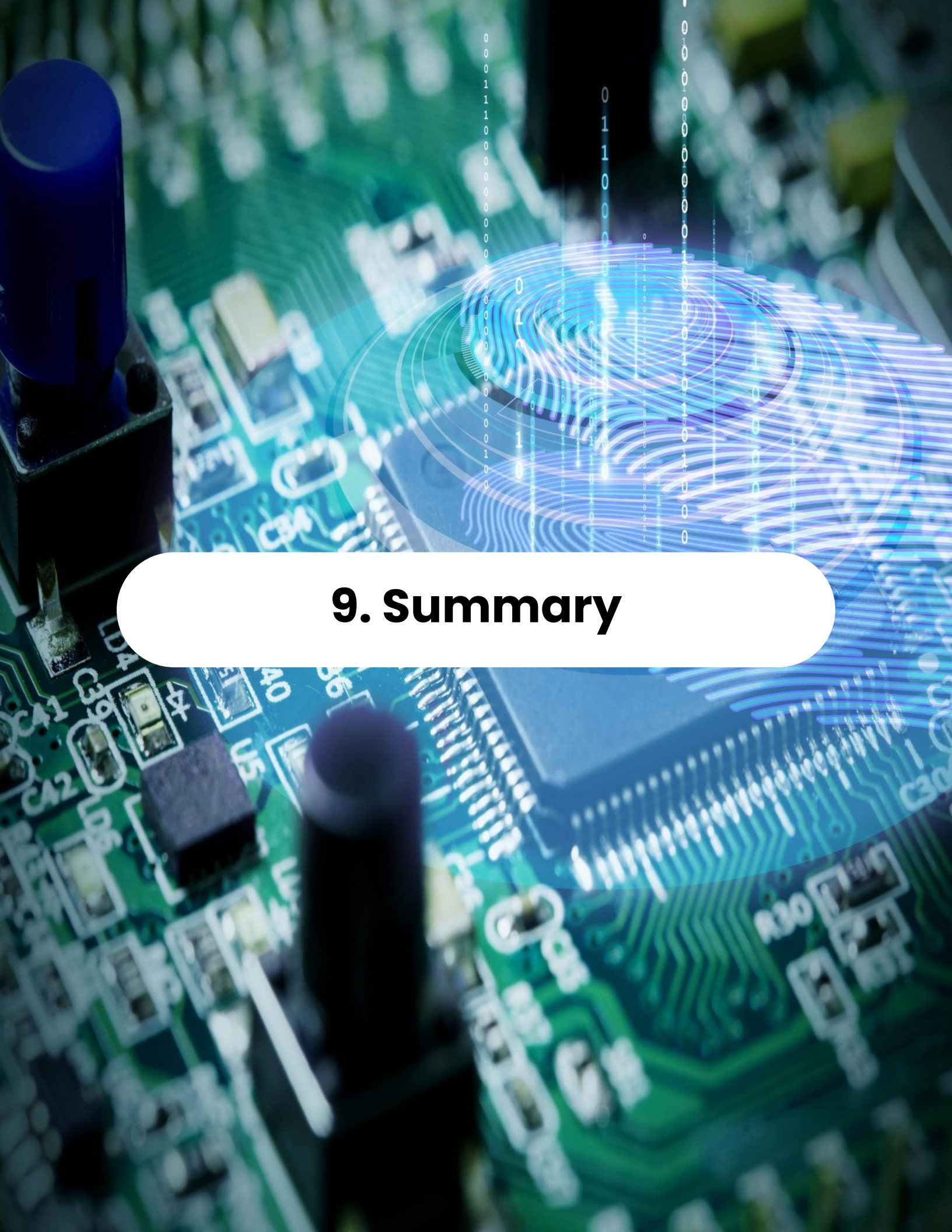
8. Tuning with menuconfig

Open the SDK Configuration editor as on Day 2 (`Ctrl+Shift+P` → **ESP-IDF: Launch Menuconfig**). Scheduling-related options include:

Menu location	Option	Effect
Component config → FreeRTOS → Kernel	Run FreeRTOS only on first core	Single-core mode (<code>CONFIG_FREERTOS_UNICORE</code>)
Component config → FreeRTOS → Kernel	<code>configTICK_RATE_HZ</code>	Tick frequency (default 100 Hz)
Component config → ESP System Settings	Main task core affinity	Which core runs <code>app_main()</code>

Experiment: Raise `configTICK_RATE_HZ` to 1000, rebuild, and notice that `pdMS_T0_TICKS(1)` now equals 1 tick. Higher tick rates give finer timing but cost more CPU time in tick interrupts.

Careful: Changing kernel settings affects the whole system. Change one option at a time and re-test.



9. Summary

9. Summary

Today, you learned:

- The scheduler decides which Ready task runs, using **fixed priorities with preemption and time slicing**
- Tasks are **Running, Ready, Blocked, or Suspended**; Blocked tasks use no CPU
- Each core runs an **Idle task**; never starve it or the watchdog will fire
- The ESP32 runs **one FreeRTOS on two cores (SMP)**; Core 0 hosts most system work by default
- `xTaskCreate()` makes **unpinned** tasks; `xTaskCreatePinnedToCore()` fixes the core
- `xPortGetCoreID()` shows where code is running **right now**
- Pin only with a reason, and write **portable** code for single-core chips

9. Key Takeaways at a Glance

Concept	Remember
Scheduling policy	Highest priority first; equal priorities share time each tick
Tick	100 Hz by default → 1 tick = 10 ms
Priorities	0 (idle) to 24; higher number wins
Task states	Running, Ready, Blocked, Suspended
Cores	Core 0 (PRO_CPU), Core 1 (APP_CPU) — dual-core chips only
Unpinned creation	<code>xTaskCreate()</code> / <code>tskNO_AFFINITY</code>
Pinned creation	<code>xTaskCreatePinnedToCore(..., coreID)</code>
Which core?	<code>xPortGetCoreID()</code>

You now understand **how and where** your tasks run — the foundation for queues, semaphores, and synchronization.



10. Challenge for Today

10. Challenge for Today

Modify the example to:

- Create **three** tasks: one pinned to **Core 0**, one pinned to **Core 1**, and one **unpinned**
- Use **one shared task function** and pass each task its label and period through `pvParameters`
- Give them different print periods, e.g., **500 ms, 1000 ms, and 2000 ms**
- Print the task name, core ID, priority, and tick count
- Use `CONFIG_FREERTOS_UNICORE` so the project also builds on single-core chips

Example output:



```
[tick  50] Core0-Task | core=0 | prio=5
[tick 100] Core1-Task | core=1 | prio=5
[tick 100] Core0-Task | core=0 | prio=5
[tick 200] Free-Task  | core=1 | prio=5
```

Useful functions: `pcTaskGetName(NULL)` , `uxTaskPriorityGet(NULL)` , `xTaskGetTickCount()` .

10. Challenge Hints

A starting skeleton (fill in the TODOs):

```
1 #include <stdint.h>
2
3 typedef struct {
4     const char *label;
5     uint32_t    period_ms;
6 } task_cfg_t;
7
8 static const task_cfg_t cfg_core0 = { "Core0-Task", 500 };
9 // TODO: cfg_core1, cfg_free
10
11 static void worker_task(void *pvParameters) {
12     const task_cfg_t *cfg = (const task_cfg_t *)pvParameters;
13     while (1) {
14         // TODO: print tick, name, core, priority
15         vTaskDelay(pdMS_TO_TICKS(cfg->period_ms));
16     }
17 }
```

- Make the config structs `static const` (or global): the task keeps using the pointer after `app_main()` returns
- **Bonus:** change one task's priority and observe the effect; try 1000 Hz tick rate